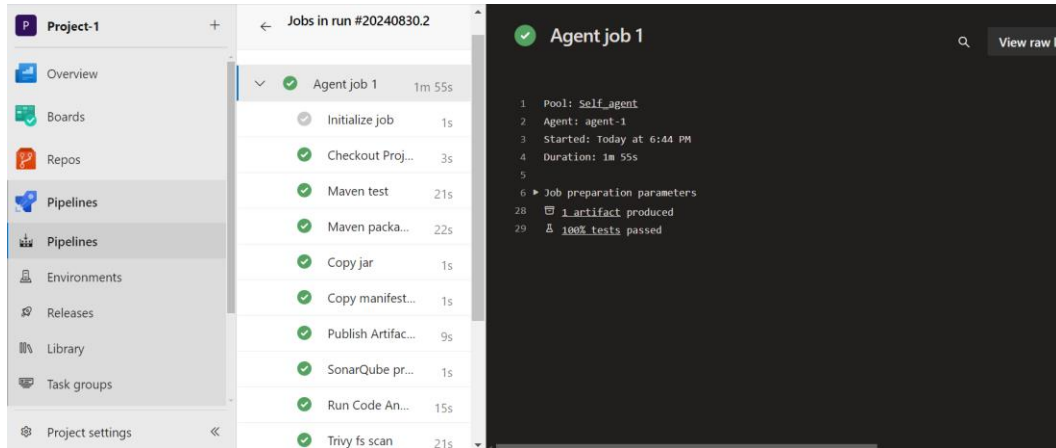


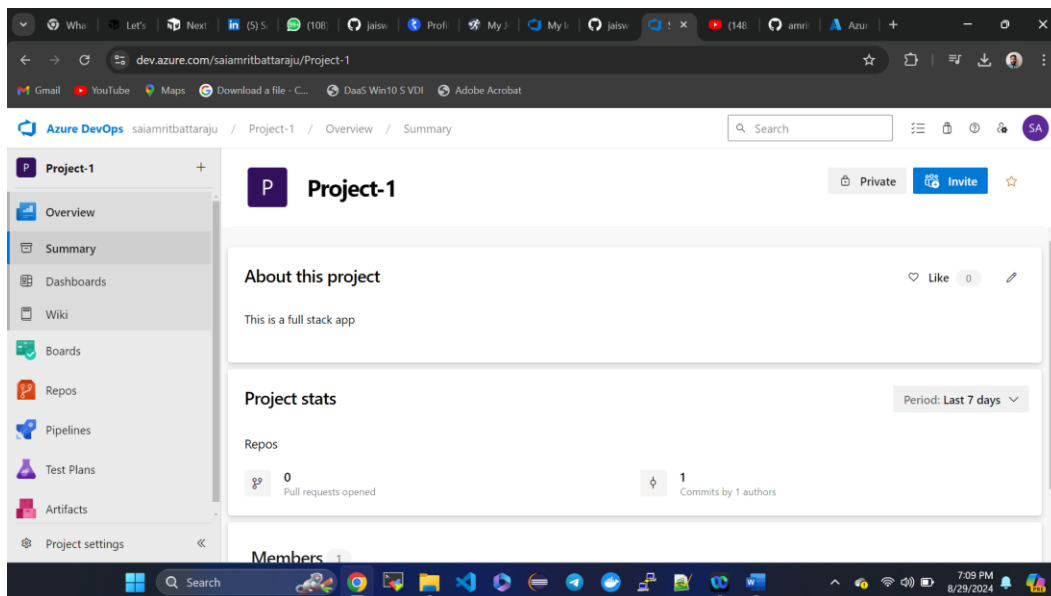
## Azure DevOps CICD pipeline

First create azure devops platform using azure devops organization.

Organization is nothing but having multiple projects which will further have their own cicd pipeline



Now Lets create project a project called Project-1.



First Thing we have to do is import the git repository by clicking on the Repos in left hand side.

The Best part of azure devops is everything is integrated like pipeline, artifact, repo, etc.

Step 1: lets import our repo in our demo project.

Note: In Azure devops there are 2 types of pipeline

- 1> Classic pipeline
- 2> Yaml pipeline

By default the classic pipeline is disabled in azure devops so lets enable them.

To enable the classic pipeline go to your organization>organization settings>pipeline tab>settings || turn off the options **Disable creation of classic build pipelines** & **Disable creation of classic release pipelines**



Off

### Disable creation of classic build pipelines

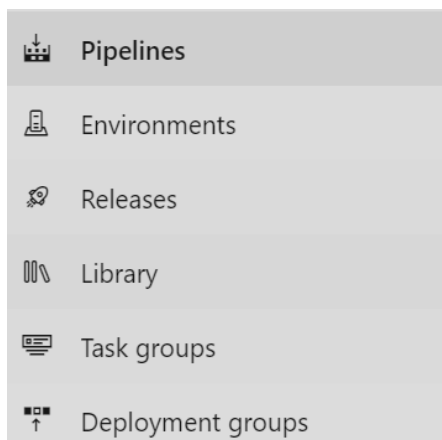
No classic build pipelines can be created / imported. Existing c



Off

### Disable creation of classic release pipelines

No classic release pipelines, task groups, and deployment groups will continue to work.



Following will be enabled in the project.

new pipeline

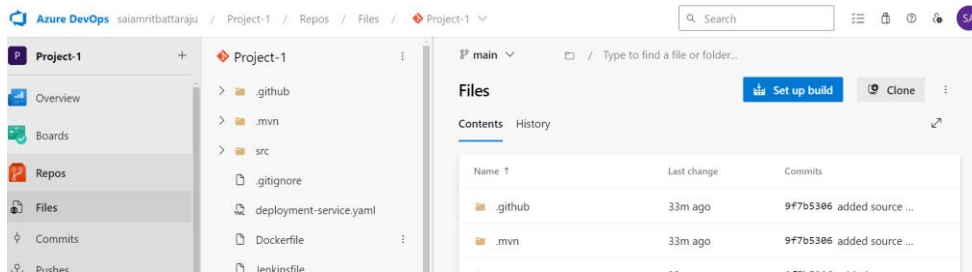
## Where is your code?

-  **Azure Repos Git** YAML  
Free private Git repositories, pull requests, and code search
-  **Bitbucket Cloud** YAML  
Hosted by Atlassian
-  **GitHub** YAML  
Home to the world's largest community of developers
-  **GitHub Enterprise Server** YAML  
The self-hosted version of GitHub Enterprise
-  **Other Git**  
Any generic Git repository
-  **Subversion**  
Centralized version control by Apache

Use the [classic editor](#) to create a pipeline without YAML.

And in the bottom we can see there is classic editor option enabled.

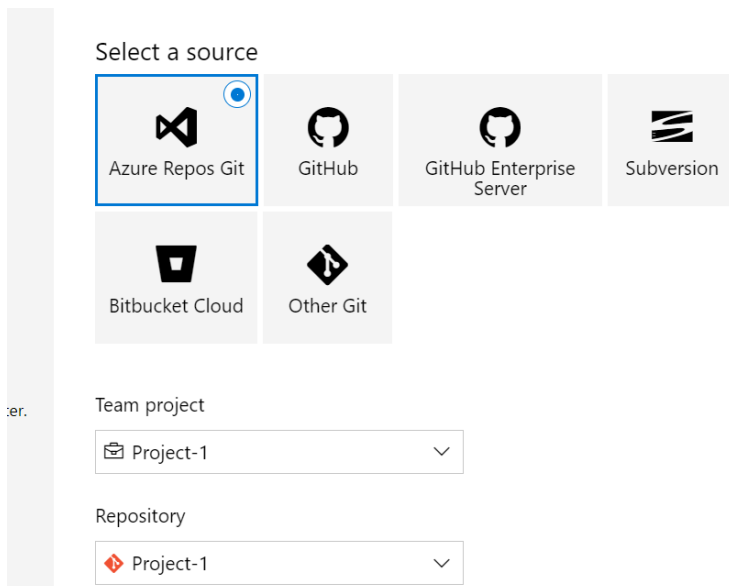
Now import the git repo into azure repo under repos sections here is the below screen shot.



## Step 2:

Click on pipeline on the left side and create new pipeline.

The click on [Use the classic editor](#) to create a pipeline without YAML. At the end of the page.



As we had imported the repo to the azure it would reflect under Azure repos git option . (above screen shot)

Now we have to select what kind of project is ours like maven docker Kubernetes etc , in my case it is maven project so select maven project.

## Select a template

Or start with an  Empty job

 Search



### Azure web app for ASP.NET

Build, package, test, and deploy an ASP.NET Azure Web App.



### Docker container

Build a Docker image and push it to a container registry.

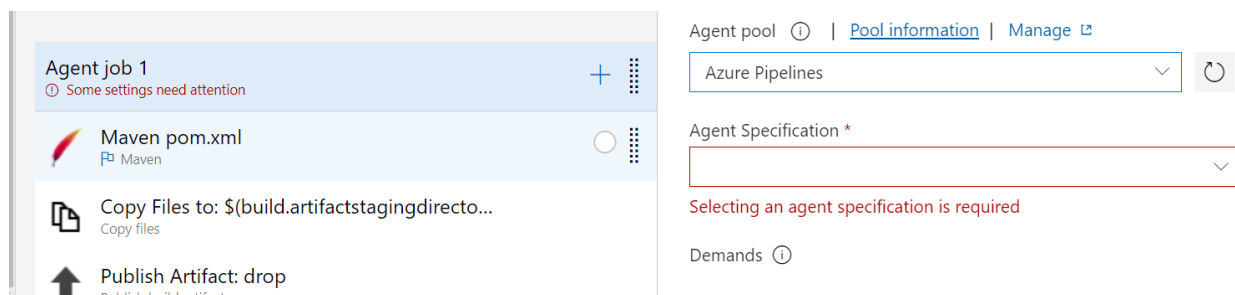


### Maven

Build and test a Java project with Apache Maven.

Apply

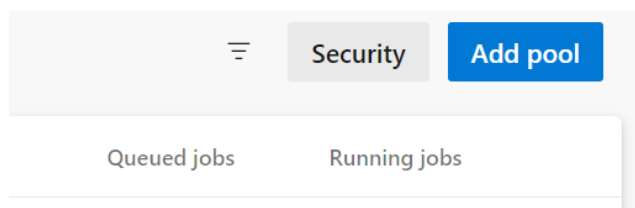
Once we click on apply we have list of option to choose from.



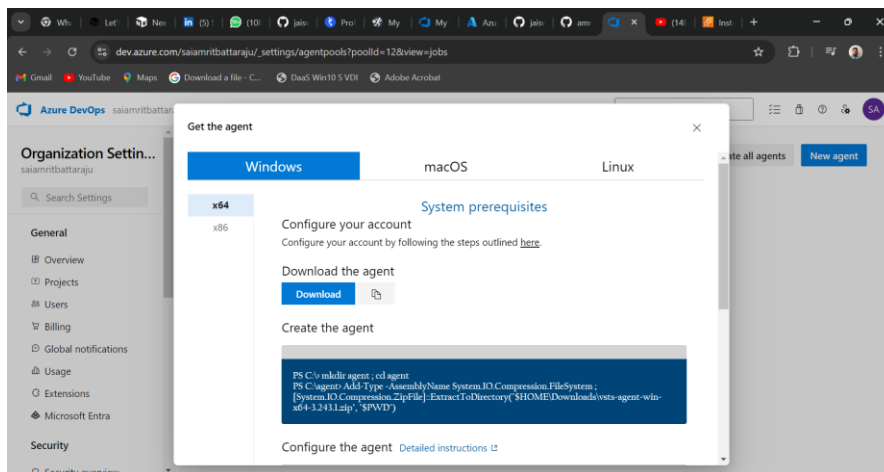
The screenshot shows the Azure Pipelines console. On the left, a list of jobs is visible: 'Agent job 1' (with a warning icon and text 'Some settings need attention'), 'Maven pom.xml' (with a Maven icon), 'Copy Files to: \$(build.artifactstagingdirecto...)' (with a file icon), and 'Publish Artifact: drop' (with an upload icon). On the right, the configuration for 'Agent job 1' is shown. It includes an 'Agent pool' dropdown set to 'Azure Pipelines', an 'Agent Specification' dropdown (which is empty and has a red border with the error message 'Selecting an agent specification is required'), and a 'Demands' section.

Here azure gives the free agent pools which needs access for running the pipeline so instead we will create one ec2 instance in aws and add that ec2 in as pipeline here.

So here in order to add ec2 as agent there are 2 options: project setting > pipeline > agent pool > give agent name > add the IP address as agent. This will be specific to only the particular project, but if you want to add agent to all the projects, we can go to the organization setting > pipeline > agent pool > agent name > add IP address. Select as self-hosted agent and proceed for adding the agent as node inside the agent.

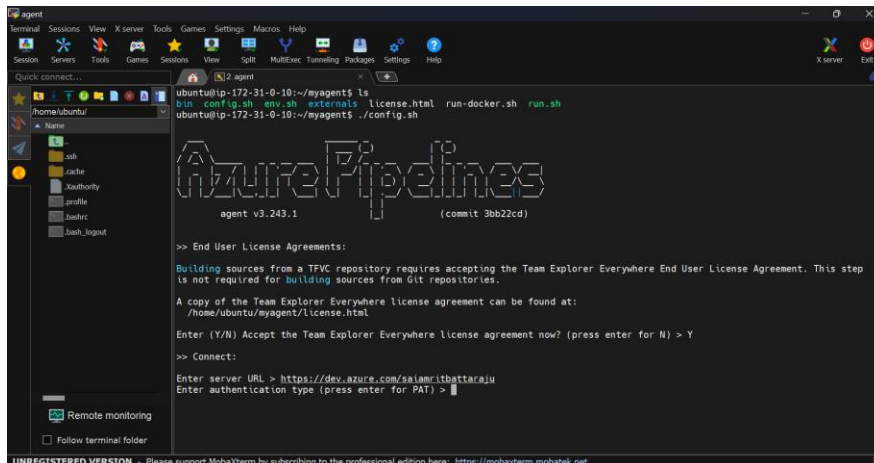


The screenshot shows the top of the Azure Pipelines console. It includes a hamburger menu icon, a 'Security' button, and a blue 'Add pool' button. Below these, there are tabs for 'Queued jobs' and 'Running jobs'.



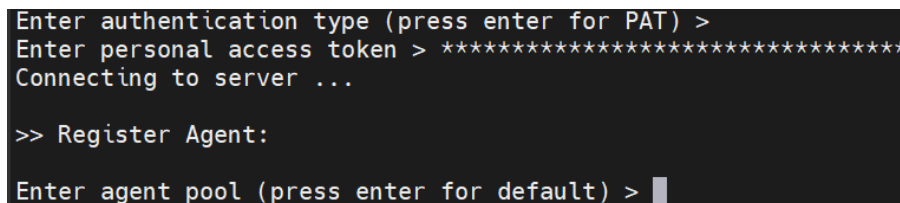
In order to add we have download the package in our virtual machines like list cmds to be executed a= in virtual machine to be added as agent.

With those commands one tar folder will be created and under that we have to run 2 file one is confi.sh and run.sh but when we run config.sh it will ask for server url and token to connect to our azure organization below are the screen shots



Create token under account settings

| Token name                                 | Organization     | Status |
|--------------------------------------------|------------------|--------|
| <input type="checkbox"/> Agent Full access | saamritbattaraju | Active |



Here you have to give agent pool name and agent name and after that agent will be added but it will be offline as we haven't run the run.sh

|                      |      |         |
|----------------------|------|---------|
| agent-1<br>● Offline | Idle | 3.243.1 |
|----------------------|------|---------|

```
Self agent
Enter agent name (press enter for ip-172-31-0-10) > agent-1
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2024-08-29 14:58:04Z: Settings Saved.
ubuntu@ip-172-31-0-10:~/myagent$ ./run.sh
Scanning for tool capabilities.
Connecting to the server.
2024-08-29 15:00:16Z: Listening for Jobs
```

|                     |      |         |
|---------------------|------|---------|
| agent-1<br>● Online | Idle | 3.243.1 |
|---------------------|------|---------|

Now let us install all the required tools on our virtual machine.

```
sudo apt install openjdk-17-jre-headless
```

```
sudo apt install maven -y
```

Sonarqube [here we will use sonar qube developer edition which will check code quality on multiple branches where as community edition will cover only for single branch to use that we using cutom docker image which has preinstall developer edition sonarqube image]

```
sudo snap install trivy
```

```
sudo apt install docker.io -y (by default ubuntu user will not have access to execute the docker commands for that run below cmd)
```

```
sudo chmod 666 /var/run/docker.sock (or) sudo usermod -aG docker ubuntu
```

```
newgrp docker
```

now installsonar qube

```
sudo docker run -d --name sonarqube -p 9000:9000 mc1arke/sonarqube-with-community-branch-plugin
```

now try to access sonar qube. (default user name and password admin:admin)

As we have installed java, maven and other tools we have to define them in capabilities under agent which we created

Jobs
Capabilities

User-defined capabilities
+

≡

No user-defined capabilities
[Add a new capability](#)

Click on add capabilities.

### Add user-defined capability

This action allows the agent to run jobs with a matching user-defined demand in a pipeline.

Name:

maven

Value:

true

### Add user-defined capability

This action allows the agent to run jobs with a matching user-defined demand in a pipeline.

Name:

java

Value:

true

Now again start the agent by ./run.sh under cd /home/ubuntu/myagent

Now lets do a dry run check by adding maven goals which are

Mvn test and mvn package add these goals by cloning the task and then click on save and queue

| ✓ | Agent job 1        | 1m 6s |
|---|--------------------|-------|
| ✓ | Initialize job     | 3s    |
| ✓ | Checkout Proj...   | 3s    |
| ✓ | Maven test         | 26s   |
| ✓ | Maven packa...     | 21s   |
| ✓ | Copy Files to: ... | 1s    |
| ✓ | Publish Artific... | 8s    |
| ✓ | Post-job: Ch...    | <1s   |

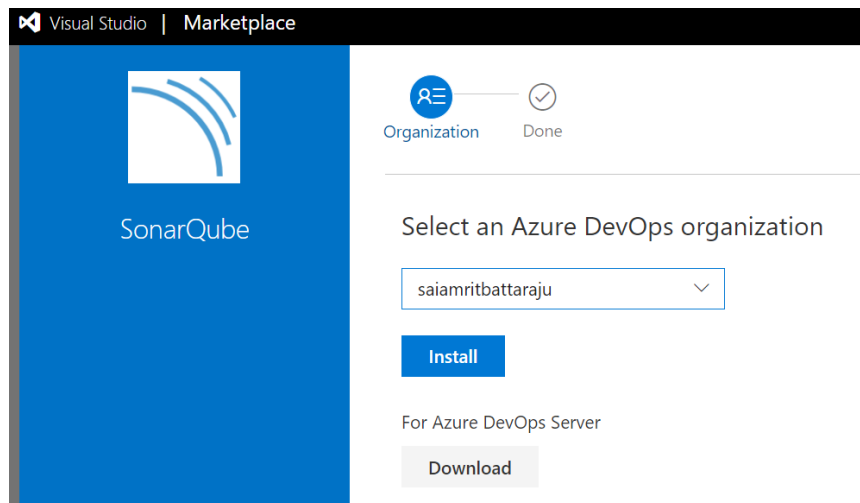
```
1 Pool: Self_agent
2 Agent: agent-1
3 Started: Today at 9:46 PM
4 Duration: 1m 6s
5
6 ▶ Job preparation parameters
28 ▶ fx 1 queue time variable used
31 📦 1 artifact produced
32 🏆 100% tests passed
```

Which got success now we can see the artifact as well which can store locally as well

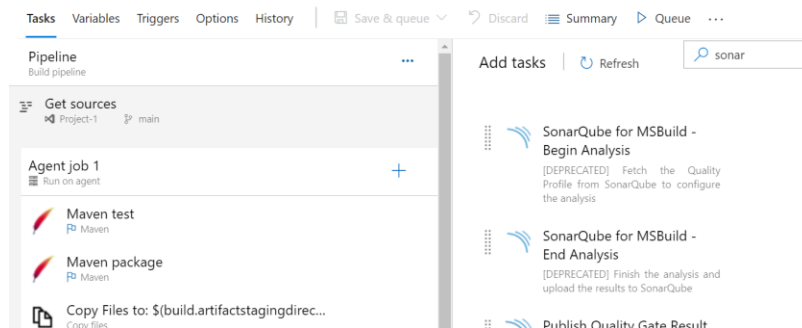
We are going to integrate different things

Now in order to integrate sonarqube in azure just like in Jenkins as we have plugins same thing we have one thing called azure market place which are called extensions in azure.

Go to market place search for sonarqube click on get it free and install on azure organization.



Then go to the pipeline and click on edit pipeline and click on plus symbol and search sonarqube which we installed.



First add Prepare analysis on SonarQube

Display name \*

SonarQube Server Endpoint \* ⓘ | [Manage](#)

ⓘ This setting is required.

Under that we have to add our sonarqube endpoint by clicking on new button



## New service connection



### Server Url

http://13.235.79.153:9000

Url for the SonarQube Server to connect to.

### Authentication

### Token

.....

Authentication Token generated through SonarQube (go to My Account > Security > Generate Tokens)

### Details

#### Service connection name

son-con

#### Description (optional)

son-con

And chose Choose the way to run the analysis

Use standalone scanner.

Mode (select anyone if 1<sup>st</sup> option properties should in our repo is 2<sup>nd</sup> option we can give manually)

And under advanced section we have to give where our sonar has to search for jar file

☒ Manually provide configuration

Project Key \* ⓘ

Boardgame

Project Name ⓘ

Boardgame

Project Version ⓘ

1.0

Sources directory root \* ⓘ

.

## Advanced ^

### Additional Properties ⓘ

```
# Additional properties that will be passed to the scanner
# Put one key=value per line, example:
# sonar.exclusions=**/*.bin
sonar.java.binaries=.
```

Can give sonar.java.binaries=target

Now add another stage by clicking on plus symbol

Add Run Code Analysis.

Now add trivy as we had already installed trivy we can search for cmd and give trivy command as task

Give command trivy fs --format table -o trivy-fs-report.html

Now just do a dry run check.

|   |                    |        |
|---|--------------------|--------|
| ✓ | Agent job 1        | 1m 40s |
| ✓ | Initialize job     | 2s     |
| ✓ | Checkout Proj...   | 3s     |
| ✓ | Maven test         | 19s    |
| ✓ | Maven packa...     | 20s    |
| ✓ | Copy Files to: ... | 1s     |
| ✓ | Publish Artific... | 8s     |
| ✓ | SonarQube pr...    | 1s     |
| ✓ | Run Code An...     | 15s    |
| ✓ | Trivy fs scan      | 26s    |
| ✓ | Post-job: Ch...    | <1s    |

```
1 Pool: Self_agent
2 Agent: agent-1
3 Started: Today at 10:30 PM
4 Duration: 1m 40s
5
6 ▶ Job preparation parameters
28 ▶ fx 1 queue time variable used
31 📦 1 artifact produced
32 🟢 100% tests passed
```

Now add docker as new task which docker build and push where we will get some initial errors we just have to refresh the page multiple times it will be fixed.

Container Repository ^

Container registry ⓘ | [Manage](#)

docker-con ▼ ↻ + New

Container repository ⓘ

amrith028/boardgame

Commands ^

Command \* ⓘ

buildAndPush ▼

Same like sonarqube add and give docker credentials and give tag to the image after image is pushed

Create another image and for image scan clone the trivy task and give the below command

Give command `trivy image --format table -o trivy-fs-report.html amrith028/boardgame:latest`

NOW lets create AKS cluster for deployment.

Just search and click on AKS and create cluster for dev/test environment.

Select all the required option create the AKS cluster, once the cluster is up we have to create release pipeline which will on left side we will create it in such a way that if any changes automatically it should trigger the ci and cd pipeline.

Now create cd pipeline which are release on left hand side.

The screenshot shows the Azure DevOps 'New release pipeline' interface. On the left, a sidebar lists 'Overview', 'Boards', 'Repos', 'Pipelines', 'Environments', and 'Releases'. The main area is titled 'All pipelines > New release pipeline' and has tabs for 'Pipeline', 'Tasks', 'Variables', 'Retention', and 'Options'. Below these tabs, there's a '+ Add' button and a 'Stages | + Add' dropdown. A 'Stage 1' box is visible with a 'Select a template' button. On the right, a 'Select a template' section offers an 'Empty job' option. Below that, a 'Featured' section lists three templates: 'Azure App Service deployment', 'Deploy a Java app to Azure App Service', and 'Deploy a Node.js app to Azure App Service'.

Click on empty job to create.

In azure to deploy pods we need to give the path of our manifest file which will deploy the pods for that we have to add one more step in ci pipeline to storing the manifest file.

Maven package  
Maven

Copy jar  
Copy files

Copy manifest file  
Copy files

Publish Artifact: drop  
Publish build artifacts

Display name \*

Copy manifest file

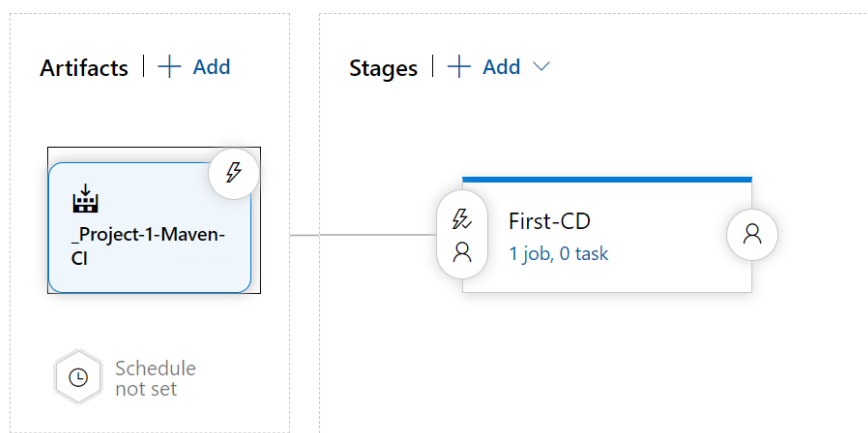
Source Folder ⓘ

\$(system.defaultworkingdirectory)

Contents \* ⓘ

deployment-service.yaml

Now select the artifact for the release cd pipeline



Now on the above screen click on job and task add the agent and give it a name as deploy to k8

Deploy to K8  
Deployment process

Agent job  
Run on agent

Agent job ⓘ

Remove

Display name \*

Agent job

Agent selection ^

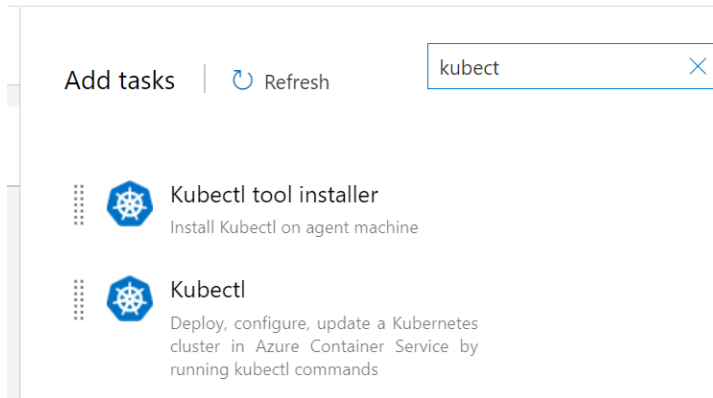
Agent pool ⓘ | [Pool information](#) | [Manage](#)

Self\_agent ▾

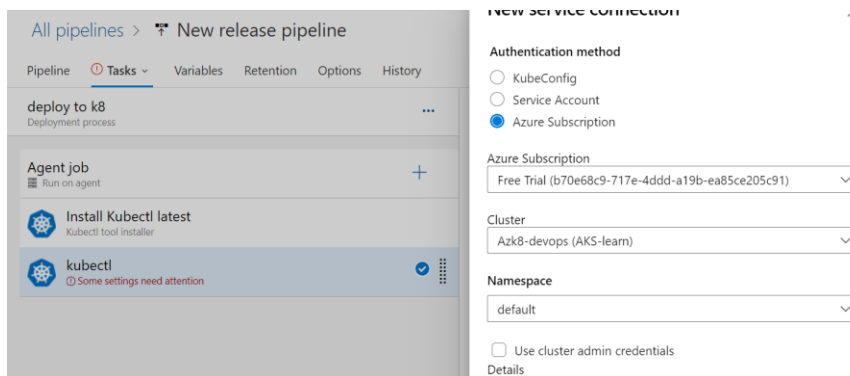
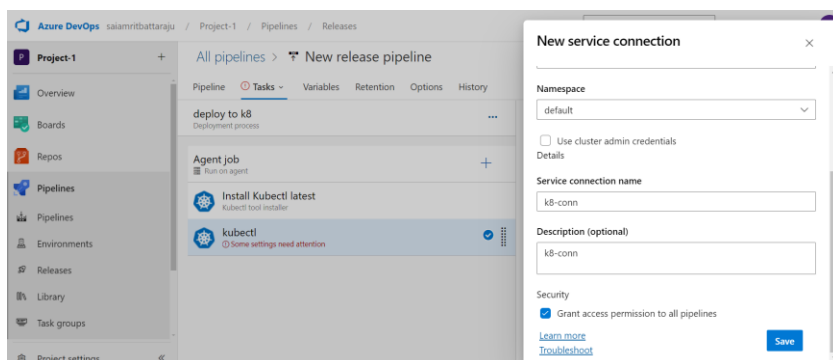
↺

Demands ⓘ

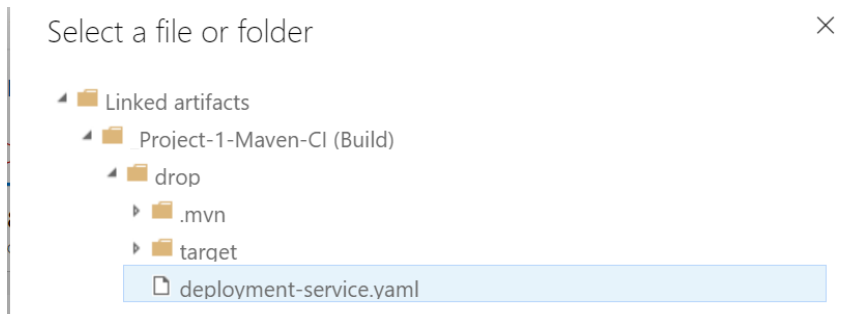
Now as we don't have kubectl we have to install on our server for that on cd pipeline we can add kubectl installer and kubectl



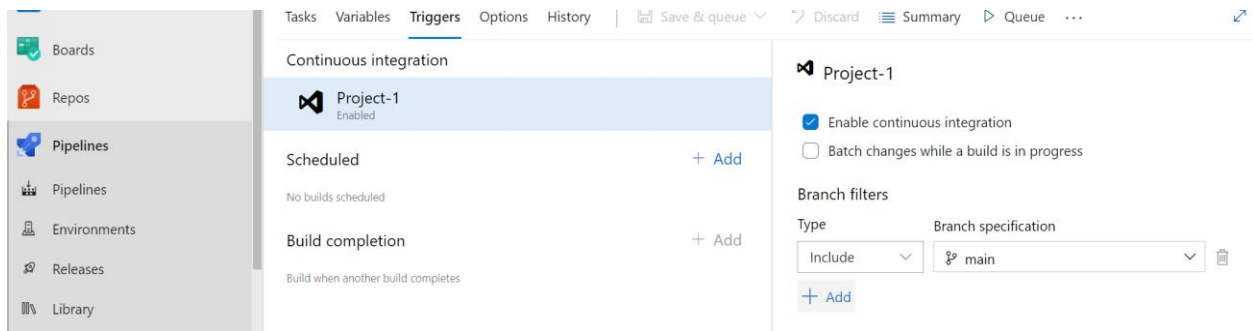
Now we can add the cluster which we created here under kubect!



Then in bottom click on set up connections once the setup is done we can give the file path of our manifest file like where is that file which we created in ci pipeline for that pop up we have to run the pipeline once as we added the new step for manifest file to save as artifact

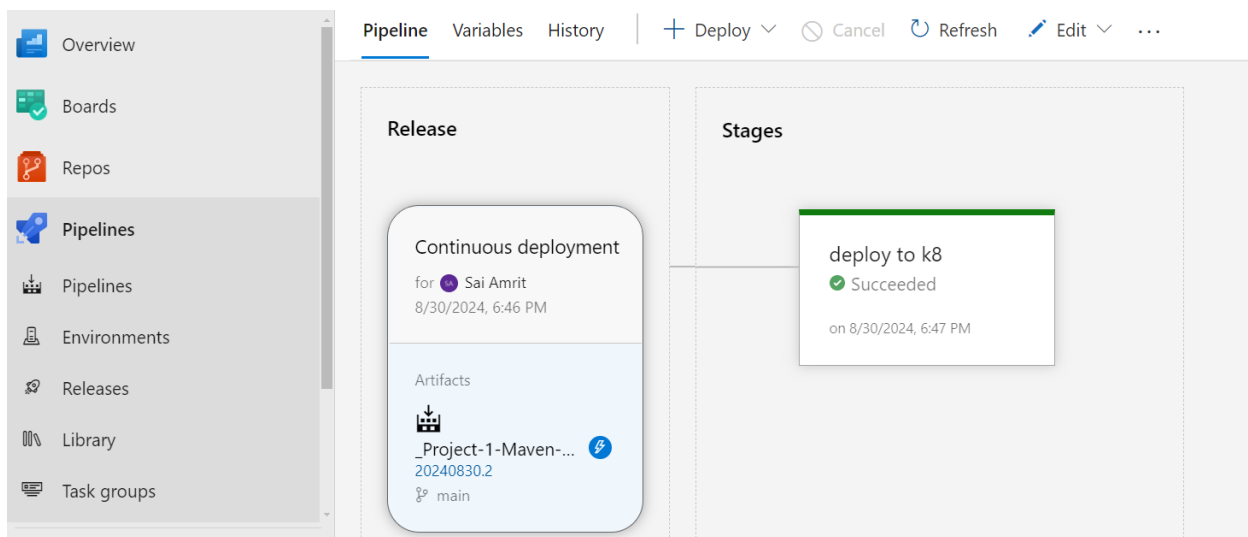
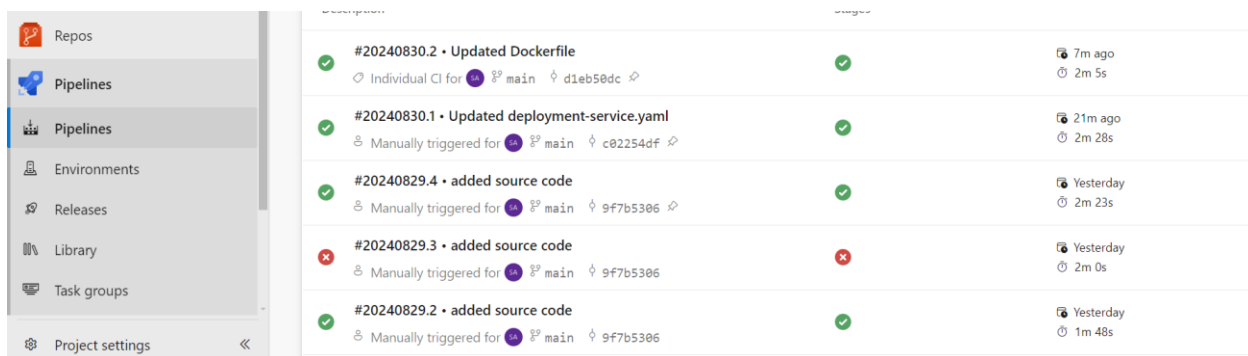


Now to automate the both ci and cd at a time we can enable triggers in ci pipeline



With that if there is any change in the repo it will automatically trigger the ci and cd pipeline.

Now trigger the pipeline which will auto run ci and cd both.



Now to access the pod under cluster ip under workloads of the aks Cluster

Tags

Diagnose and solve problems

Microsoft Defender for Cloud

Cost analysis

Kubernetes resources

Namespaces

Workloads

Services and ingresses

Storage

Services

Ingresses

Filter by service name

Filter by namespace

Enter the full service name

All namespaces

Add label filter

| <input type="checkbox"/> | Name                           | Namespace   | Status | Type         | Cluster IP   | External IP                  | Ports         |
|--------------------------|--------------------------------|-------------|--------|--------------|--------------|------------------------------|---------------|
| <input type="checkbox"/> | <a href="#">kubernetes</a>     | default     | ✔ Ok   | ClusterIP    | 10.0.0.1     |                              | 443/TCP       |
| <input type="checkbox"/> | <a href="#">kube-dns</a>       | kube-system | ✔ Ok   | ClusterIP    | 10.0.0.10    |                              | 53/UDP,53/TCP |
| <input type="checkbox"/> | <a href="#">metrics-server</a> | kube-system | ✔ Ok   | ClusterIP    | 10.0.51.230  |                              | 443/TCP       |
| <input type="checkbox"/> | <a href="#">boardgame-svc</a>  | default     | ✔ Ok   | LoadBalancer | 10.0.101.207 | <a href="#">4.156.93.167</a> | 80:30153/TCP  |

Welcome to BoardGame Database! 🎲

Home Login Sign-up

Boardgame Lists

Splendor

Clue

Linkee

For more services, login [Here](#)

To join to the service, [Click](#) here